Esempi su cicli e istruzioni di selezione

Marco Cerbella

Lezione 17 – 9 novembre 2026

1 Perché i cicli sono importanti

Un *ciclo* è una delle strutture logiche di base della programmazione: definire un ciclo permette al calcolatore di eseguire lo stesso compito ripetutamente. Ogni volta che nella descrizione in linguaggio naturale delle specifiche del programma compaiono le parole «ogni» o «per ciascuno» serve un ciclo. Anche «per sempre» corrisponde a un ciclo: `while(1)`.

2 Esempi sui cicli

2.1 Condizioni logiche: = e ==

```
int x = 1;
while (x = 1) {          // ERRORE: assegnamento, non confronto
    //...
    printf("Insert 1 to continue");
    scanf("%d", &x);
}
```

Se si usa un solo segno di uguale per controllare l'uguaglianza, il programma *assegna* il valore a destra alla variabile a sinistra, e il risultato dell'espressione è il valore assegnato. In questo caso il valore è sempre 1, che viene trattato come sempre vero: il ciclo non termina mai. La versione corretta è

```
int x = 1;
while (x == 1) {
    //...
    printf("Insert 1 to continue");
    scanf("%d", &x);
}
```

2.2 Trasformare i cicli

Uno stesso ciclo si può scrivere con `for`, `while` o `do...while`:

```
int n = 4;
for (int i = 0; i < n; i++) {
    printf("OK\n");
}
```

```c
int n = 4;
int i = 0;
while (i < n) {
    printf("OK\n");
    i++;
}
```

```c
int n = 4;
int i = 0;
do {
    printf("OK\n");
    i++;
} while (i < n);
```

2.3 do...while

Il corpo di un `do...while` viene eseguito *almeno una volta*:

```c
#include <stdio.h>
int main() {
    double number, sum = 0;
    // il corpo del ciclo viene eseguito almeno una volta
    do {
        printf("Enter a number: ");
        scanf("%lf", &number);
        sum += number;
    } while (number != 0.0);
    printf("Sum = %.2lf", sum);
    return 0;
}
```

2.4 Scorrere un array

```c
#include <stdio.h>
int main() {
    int foo[] = {16, 2, 77, 40, 12071};
    int n, result = 0;
    for (n = 0; n < 5; ++n) {
        result += foo[n];
    }
    printf("%d", result);      // stampa 12206
    return 0;
}
```

La somma $16 + 2 + 77 + 40 + 12071$ dà 12206.

Osservazione. Nelle slide l'esempio includeva `<stdlib.h>` ma usava `printf`; qui è stato corretto con `<stdio.h>`.

2.5 Scorrere una matrice

```c
const int width = 5;
const int height = 3;
int main() {
    int mat[height][width];
    for (int n = 0; n < height; n++)
        for (int m = 0; m < width; m++)
            mat[n][m] = (n + 1) * (m + 1);
}
```

2.6 Array come argomenti di funzione

Quando il nome di un array compare come argomento di una funzione, il compilatore lo converte implicitamente in un puntatore al primo elemento dell'array. Perciò `int name[]` e `int *name` sono equivalenti:

```c
#include <stdio.h>
void fun(int a[], int n) {
    for (int i = 0; i < n; ++i)
        a[i] = i;
}
int main() {
    int a[3];
    fun(a, 3);
}
```

```c
#include <stdio.h>
void fun(int* a, int n) {
    for (int i = 0; i < n; ++i)
        *(a + i) = i;
}
int main() {
    int a[3];
    fun(a, 3);
}
```

In C non esistono davvero «variabili array»: si lavora in realtà con puntatori, con una sintassi alternativa.

2.7 Passare una matrice

```c
#include <stdio.h>
int matrix_sum(int row, int column, int m[row][column]) {
    int sum = 0;
    for (int i = 0; i < row; i++)
        for (int j = 0; j < column; j++)
            sum += m[i][j];
    return sum;
}
```

```c

int main() {
    int m[2][3] = {1, 4, 5, 2, 1, 1};
    int result;
    result = matrix_sum(2, 3, m);
    printf("%d", result);      // stampa 14
}
```

La matrice 2×3 contiene per righe $\begin{smallmatrix} 1 & 4 & 5 \\ 2 & 1 & 1 \end{smallmatrix}$, e la somma dei suoi elementi è 14.

2.8 Un esempio avanzato: cicli annidati (bubble sort)

```c
#include <stdio.h>
int main() {
    int n, c, d, swap;
    printf("Enter number of elements\n");
    scanf("%d", &n);
    printf("Enter %d integers\n", n);
    int array[n];
    for (c = 0; c < n; c++)
        scanf("%d", &array[c]);
    for (c = 0; c < (n - 1); c++)
    {
        for (d = 0; d < n - c - 1; d++)
        {
            if (array[d] > array[d+1])
            {
                swap       = array[d];
                array[d]   = array[d+1];
                array[d+1] = swap;
            }
        }
    }
    return 0;
}
```

Il ciclo esterno esegue $n-1$ passate; a ogni passata il ciclo interno confronta gli elementi adiacenti e li scambia se sono nell'ordine sbagliato, spostando così il massimo dei rimanenti in fondo all'array.

3 Esempi sulle istruzioni di selezione

3.1 Condizioni logiche

```c
int age = 17;
if (age = 18)           // assegnamento!
    printf("OK!");
```

Il valore dell'espressione `age = 18` è 18, che in C è vero: stampa sempre «OK!» e `age` diventa 18. Invece

```c
int age = 17;
if (age == 18)
    printf("OK!");
```

stampa «OK!» solo quando `age` è uguale a 18, e `age` rimane 17.

3.2 Pari o dispari

```c
#include <stdio.h>
int main() {
    int number = 0;
    printf("Enter an integer: ");
    scanf("%d", &number);
    // vero se il resto è 0
    if (number % 2 == 0)
        printf("%d is an even integer.", number);
    else
        printf("%d is an odd integer.", number);
    return 0;
}
```

3.3 Catena di if

Una sequenza di `if` indipendenti *non* equivale a una catena `else if`. Nel primo caso tutte le condizioni vengono valutate; con `score = 95` tutte sono vere e il voto finale sarà quello dell'ultimo `if` soddisfatto:

```c
char grade = 'F';
int score = 0;
scanf("%d", &score);
if (score >= 90) grade = 'A';
if (score >= 80) grade = 'B';
if (score >= 70) grade = 'C';
if (score >= 60) grade = 'D';
printf("Your grade is %c", grade);      // con 95 stampa D!
```

Con la catena `else if` solo il primo ramo vero viene eseguito:

```c
char grade = 'D';
int score = 0;
scanf("%d", &score);
if (score >= 90)
    grade = 'A';
else if (score >= 80)
    grade = 'B';
else if (score >= 70)
    grade = 'C';
else if (score >= 60)
    grade = 'D';
printf("Your grade is %c", grade);
```

Osservazione. Nelle slide l'ultimo confronto era scritto `if (score >= 60);` con un punto e virgola finale: sarebbe un'istruzione vuota (errore comune descritto più sotto). Qui è stato tolto.

3.4 Attenzione all'annidamento

Le due forme seguenti sono *equivalenti*, perché l'`else` si associa sempre all'ultimo `if` senza `else`:

```c
if (x > 10) {
    if (x < 20)
        printf("x is between 10 and 20");
    else
        printf("x is greater than 20");
}
```

```c
if (x > 10)
    if (x < 20)
        printf("x is between 10 and 20");
    else
        printf("x is greater than 20");
```

Invece, *racchiudendo* il primo `if` interno in un blocco, l'`else` si associa all'`if` esterno:

```c
if (x > 10) {
    if (x < 20)
        printf("x is between 10 and 20");
}
else
    printf("x is less than 10");      // in realtà: x <= 10
```

3.5 Errori comuni

1. *Punto e virgola dopo la condizione.* L'istruzione vuota diventa il corpo dell'`if`, e la `printf` viene eseguita sempre:

   ```c
   if (age >= 18);                    // il ';' chiude l'if
       printf("You can drink!\n");    // stampata sempre
   ```

2. *Più istruzioni senza blocco.* Solo la prima istruzione dipende dall'`if`; la seconda viene sempre eseguita:

   ```c
   if (age >= 18)
       printf("You can drink!\n");
       printf("Age greater than 18\n");  // stampata sempre
   ```

3. *`else` con condizione.* Un `else` non può avere una condizione:

```c
if (a < b)
    { /* ... */ }
else if (a > b)
    { /* ... */ }
else (a == b)                          // ERRORE: else non ha condizione
    { /* ... */ }
```

4. *Confronti concatenati.* L'operatore `>` è associativo da sinistra a destra, quindi con `x = 15`:

```c
int x = 15;
if (20 > x > 10)                       // (20 > x) > 10  ->  1 > 10  ->
    falso
    printf("x is between 10 and 20");
```

L'espressione `20 > x` è vera (1), e poi $1 > 10$ è falso (0). La forma corretta usa un operatore logico:

```c
if (x > 10 && x < 20)
    printf("x is between 10 and 20");
```

Regole pratiche: (1) se ci sono più casi, un `if` per ciascuno; (2) se la condizione è lunga, scomporla e usare gli operatori logici.

3.6 Cicli infiniti e overflow

```c
#include <stdio.h>
#include <stdlib.h>
int main() {
    int i = 1;
    while (1) {
        i++;
        if (i < 0) {
            printf("%d\n", i);
            break;
        }
    }
}
```

Il ciclo sembra infinito, ma `i` continua a crescere fino a superare il massimo valore rappresentabile: la slide mostra l'output -2147483648. Questo è l'*overflow*. Con un `int` in complemento a due a 32 bit:

01111111 11111111 11111111 11111111	2 147 483 647
01111111 11111111 11111111 11111110	2 147 483 646
...	...
00000000 00000000 00000000 00000001	1
00000000 00000000 00000000 00000000	0
10000000 00000000 00000000 00000000	−2 147 483 648
10000000 00000000 00000000 00000001	−2 147 483 647
...	...
11111111 11111111 11111111 11111111	−1

Aggiungendo 1 al massimo 2 147 483 647 si passa al valore più negativo $-2\,147\,483\,648$.

Osservazione. Per gli interi *con segno*, lo standard C considera l'overflow comportamento non definito: l'output mostrato è quello tipico su una macchina con complemento a due, non un risultato garantito.